Waypoint

Product Overview

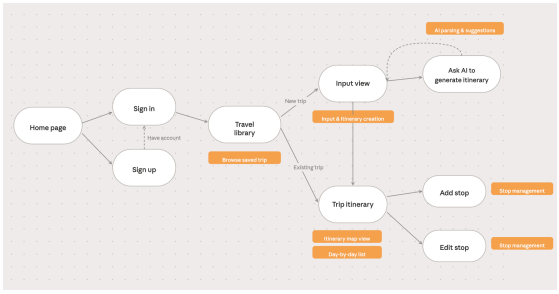
Problem Statement: Trip planning is a fragmented experience. Ideas accumulate across notes apps, group chats, browser tabs, and recommendation lists, but converting that scattered input into a usable itinerary requires significant manual effort. Existing tools require starting from scratch or locking users into rigid templates that don't reflect how people actually gather travel ideas. Waypoint solves this by accepting any form of unstructured input, notes, emails, bullet lists, or a travel blog URL, and using AI to extract, structure, and map those ideas into a day-by-day itinerary that can be edited in natural
Target Users: This application is built for casual travelers, students, and friend groups planning short domestic or local trips. The primary user has trip ideas already, a saved list, a group chat thread, a notes dump, but no efficient way to turn that into a real schedule. They are looking for a single interface that replaces all three for the purposes of trip planning. Secondary users include anyone who wants to browse and revisit saved trips across different stages: drafting, finalized, shared, or completed.
Final Feature

- Trip Input & AI Parsing: paste notes, upload a file, or import from a URL; AI extracts structured stops and plots them on a live map preview
- Itinerary View: interactive map with numbered pins and a collapsible day-by-day stop list; stops can be added, edited, reordered, and deleted
- AI Copilot: conversational panel for natural language itinerary editing; changes apply directly
- Route Optimizer: one-click reordering of stops by geographic proximity and meal timing
- Explore: curated trip starter templates and an AI-powered nearby recommendations panel
- Trip Library: all saved trips with map thumbnails, status filtering, and trip deletion
- Analytics Dashboard: trip stats, interest breakdown chart, and CSV export

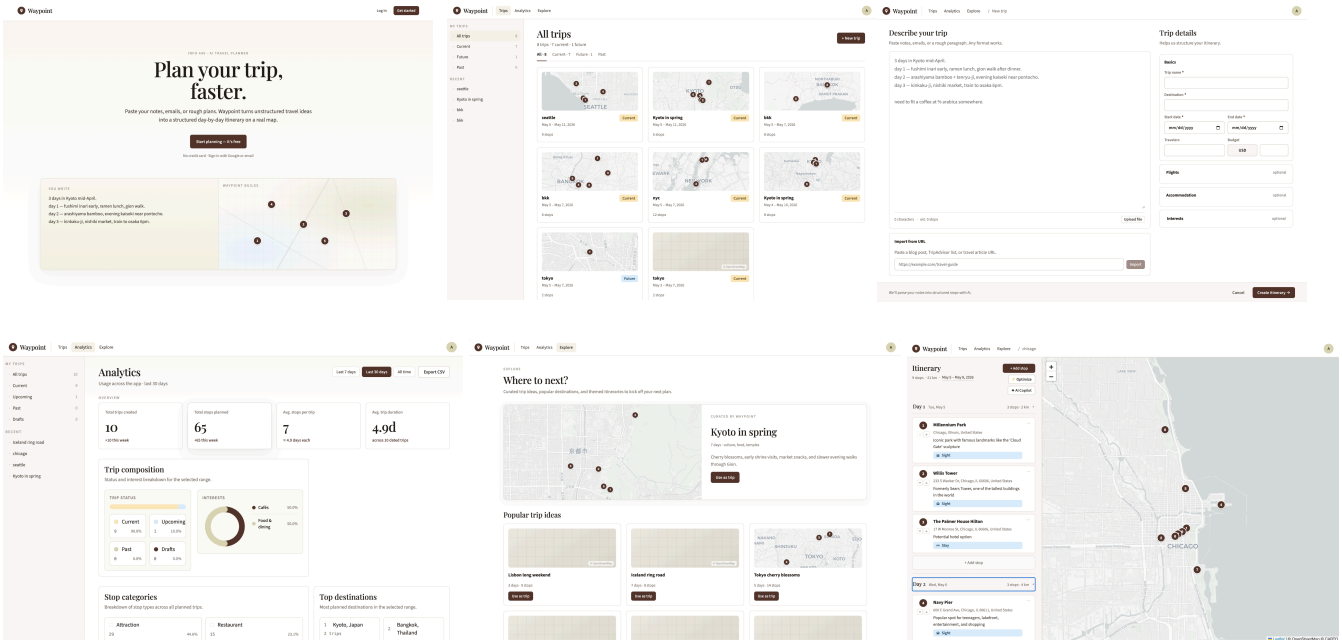
Deprioritized:

- Collaborator sharing, data layer and invite modal built, but no supported backend yet
- AI request logging: analytics panel exists but shows mock data; real logging not implemented
- Flights/accommodation fields: present in the input form but not persisted to the database

User Flow:



Updated wireframe designs:



Django System Architecture

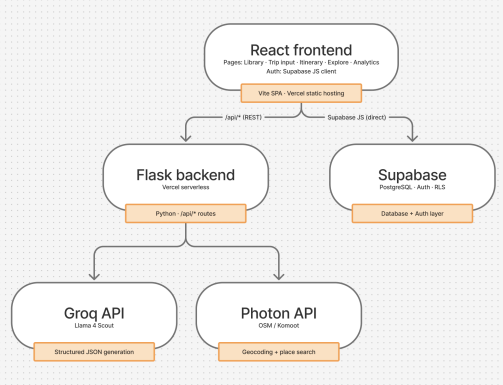
Waypoint uses Flask rather than Django. The backend is a stateless API deployed as a Vercel serverless function, a pattern that fits Flask's lightweight routing model better than Django's full project structure. All required architectural concepts are present and mapped below.

Requirement Mapping

Django Concept	Waypoint Equivalent
Models with relationships	Supabase PostgreSQL tables (trips, stops, profiles, trip_collaborators) defined in schema.sql with foreign keys and cascade deletes
Function-based views	Flask route functions in api/index.py (e.g. parse_trip, chat_itinerary, optimize_itinerary)
Templates	React components (Itinerary.jsx, Library.jsx, Analytics.jsx) with reusable shared components (Navbar, AIChatPanel, TripExplorePanel)
Forms and user input	React controlled form components: trip creation form, stop composer, date editor, AI chat input
Authentication	Supabase Auth (email/password); session managed via AuthContext.jsx; protected routes enforced in App.jsx
Internal JSON API	9 Flask endpoints all returning JSON (detailed below)

Model-driven URLs	React Router dynamic routes (/trip/:id); trip ID is the Supabase UUID primary key
Data features	Analytics page: aggregation, SVG visualizations, and CSV export sourced from live Supabase queries
Production setup	.env and .env.example for secrets, .gitignore configured, deployed on Vercel

System Architecture:



AI Integration

Where AI Enters the System:

Endpoint	Trigger	Purpose
POST /api/parse	User submits trip notes	Extract structured stops from free-form text
POST /api/import-url	User pastes a blog/article URL	Extract stops from web page content
POST /api/suggest	User requests more ideas	Generate new stops for an existing itinerary
POST /api/explore	Itinerary page loads	Recommend nearby attractions aligned with interests
POST /api/chat	User types in AI copilot panel	Conversational itinerary editing via natural language
POST /api/optimize	User clicks "Optimize"	Reorder stops for geographic efficiency and meal timing

How User Input is Processed:
All prompts use a system/user split. The system prompt defines the output schema, categories, and constraints. The user prompt provides the actual input data: notes, current stops, and conversation history. For /api/parse, the user's raw text is combined with trip context (destination, date range, interests) before being sent to the model. For /api/import-url, the fetched page has its HTML, scripts, and styles stripped before being truncated to 6,000 characters and passed to the model. For /api/chat, the last 6 messages of conversation history are included with each request to maintain context.

Which Models are Used:

Primary model: meta-llama/llama-4-scout-17b-16e-instruct via Groq.

- Speed: Groq's LPU hardware delivers consistently low latency (~1-3s for most requests), which matters for interactive UX as the user is watching the map populate in real time.
- Cost: Groq's pricing is substantially lower than GPT-4o or Claude Sonnet for equivalent output quality on structured JSON tasks.
- Output quality: Llama 4 Scout reliably follows JSON schema instructions for structured extraction with low hallucination rates on concrete place names.
- No model weights to host: Using a hosted inference API avoids GPU provisioning while still avoiding OpenAI lock-in.

Geocoding uses the Photon API (Komoot, OSM-based): free, no API key required, with global coverage and support for location-biased queries via a bounding box around the destination.

How Outputs are Generated:
JSON schema is embedded in the system prompt and the model is explicitly told to return only valid JSON with no markdown fences. Temperature is set per task: 0.2 for deterministic extraction (/api/parse, /api/optimize, /api/import-url), 0.3 for chat, 0.4 for suggestions, and 0.5 for exploration where variety is desirable.
/api/chat defines four structured action types the model returns alongside a text reply:

{"type": "add", "stop": { ...stop object... }}
{"type": "remove", "name": "exact stop name"}
{"type": "reorder", "day": 2, "order": ["Stop A", "Stop B"]}
{"type": "edit", "name": "Stop name", "updates": {"notes": "...", "day": 2}}

How Results are Returned to the User:
After the model returns stops, all coordinates are resolved via parallel Photon geocoding (max 5 workers) with a ±1.5 degree bounding box around the destination to prevent cross-continent mismatches. The final structured JSON is returned to the React frontend where stops are rendered directly onto the Leaflet map and the day-by-day stop list. For /api/chat, action objects are applied immediately to the live itinerary state without requiring a page reload.

Guardrails: If Groq fails or returns malformed JSON, /api/parse falls back to line-by-line parsing of the input text where the first 6 lines become stops. If /api/explore fails, the system returns hardcoded seed suggestions for common destinations (Tokyo, Kyoto, Osaka) or a generic template for unknown destinations. All endpoints return {'stops': [], 'warning': str(e)} on failure rather than crashing. Every AI-returned stop is normalized server-side: whitespace stripped, category validated against the allowed set, and day numbers clamped to the trip window.

What would an API-only version of our system look like?: An API-only version would use a proprietary hosted model such as GPT-4o or Claude Sonnet in place of Llama 4 Scout. The pipeline would be identical: user input goes to the model, structured JSON comes back, stops are geocoded and rendered. The difference is entirely in the model layer, closed weights, no self-hosting path, and pricing controlled entirely by the provider.

Why did we not choose that approach?:

- Cost: GPT-4o is ~$5/M input tokens vs Groq's ~$0.11/M for Llama 4 Scout, a 45x difference.
- Control: Groq supports multiple open model families; we can swap to a different Llama variant or Mixtral without changing application code.
- Privacy: User travel notes are not ingested by OpenAI's training pipelines when using open models via Groq.
- Flexibility: The same Llama 4 weights can be run locally via Ollama if scale or privacy requirements demand it, with no application code changes beyond swapping the API base URL.

Dimension	Current (Groq + Llama 4 Scout)	Fully API-Based (GPT-4o)
Cost per user/month	~$0.03-0.08	~$1.50-3.60
Latency (p50)	1.5-3s	2-5s
Output quality (JSON extraction)	High	High
Vendor lock-in	Low (open model, swappable)	High (OpenAI-specific)
Privacy	Groq ToS; no training use	OpenAI ToS
Self-hostable	Yes (Ollama + Llama)	No

What benefits did our current design provide?: The hybrid approach (open model via fast hosted inference) hits the optimal point: near-GPT-4o quality, 30-45x lower cost, and a clear path to full self-hosting if scale or privacy requirements demand it.

Evaluation & Failure Analysis

System Evaluation:

#	Input	Expected Behavior	Actual Output	Quality	Latency
1	Notes: "Day 1: Fushimi Inari hike in the morning, then Nishiki Market for lunch, Gion in the evening"	3 stops extracted, correct days/times, accurate Kyoto addresses	3 stops returned: Fushimi Inari (attraction, 08:00), Nishiki Market (restaurant, 12:00), Gion (attraction, 18:00), all geocoded correctly	High	~2.1s
2	Chat: "Move all restaurants to day 2"	All stops with category=restaurant reassigned to day 2, others unchanged	Returned edit actions for each restaurant stop updating day to 2; non-restaurant stops untouched	High	~1.8s
3	URL: TripAdvisor "Top 10 Things to Do in Osaka" article	6-10 attraction stops extracted from article body	8 stops extracted with accurate names (Dotonbori, Osaka Castle, etc.); addresses partially inferred	Medium (addresses were approximate, not geocoded to exact coordinates)	~3.4s
4	Explore: Destination "Paris", interests ["museums", "cafes"]	6-8 recommendations weighted toward museums and cafes with ratings and hours	7 recommendations returned; 5 were museum/cafe relevant, 2 were generic tourist attractions	Medium-High	~2.5s

5	Optimize: 6 stops spread across Tokyo with mixed meal/attraction categories	Stops reordered geographically per day with meals interleaved appropriately	Stops reordered with reduced geographic backtracking; explanation cited proximity and meal timing	High	~2.9s

Failure Analysis:

Failure 1: Geocoding produces wrong coordinates for ambiguous place names

- **What failed:** When a user entered "Central Park" in a trip for Sydney, Photon returned coordinates for Central Park, New York.
- **Why:** Photon's global OSM database ranks New York's Central Park higher by default. The destination bounding box (±1.5° around Sydney) was applied after the initial query, not as a hard filter.
- **Root cause:** Geocoding bias is implemented as a preference, not a constraint. Photon can still return results outside the bounding box if its relevance score is high enough.
- **Type:** Data issue, the geocoding layer, not the LLM.

Failure 2: Chat copilot generates malformed action JSON for complex multi-step requests

- **What failed:** User typed "Move the hotel to day 3 and add a dinner reservation at Nobu after it." The model returned the edit action correctly but embedded the add action as a nested key inside the first action rather than as a separate array element.
- **Why:** The system prompt specifies a flat array of action objects, but the model inferred a hierarchical structure for semantically linked actions.
- **Root cause:** Prompt ambiguity; the schema example only showed single actions, not multi-action arrays.
- **Type:** Prompt engineering issue.

Improvement: Fixing Multi-Action Chat Responses

Before: system prompt showed a single action example:

Return a JSON object with "reply" and "actions" (array).
Example actions: [{"type":"add","stop":{...}}]

After: system prompt was updated with an explicit multi-action example and a reinforcing constraint:

Return a JSON object with "reply" and "actions" (array).
Each action must be a separate object in the array, even if logically related.
Example of multi-step request:
[
{"type":"edit","name":"Grand Hyatt","updates":{"day":3}},
{"type":"add","stop":{"name":"Nobu","category":"restaurant","day":3,...}}
]
Never nest actions inside other actions.

Why it helped: Explicit counter-examples in the prompt ("never nest actions") directly address the failure mode. After the update, multi-step requests were parsed correctly in all tested cases. This is a well-known prompt engineering pattern: showing what NOT to do is often more effective than only showing correct examples.

Cost & Production Readiness

Cost and Resource Awareness:

Component	Cost Model	Estimated Cost per Active User/Month
Groq API (Llama 4 Scout)	~$0.11/M input tokens, ~$0.34/M output tokens	~$0.03-0.08 (avg 5 AI interactions, ~2K tokens each)
Photon Geocoding	Free (OSM-based, no rate limit for reasonable use)	$0.00
Supabase (database + auth)	Free tier covers ~500MB + 50K MAU	$0.00 on free tier
Vercel (serverless hosting)	Free tier covers 100GB-hrs/month	$0.00 on free tier

Estimated total: ~$0.03-0.08 per active user per month at current usage patterns.

Comparison with a fully API-based alternative (GPT-4o):

Component	Cost per Active User/Month
OpenAI GPT-4o	~$1.40-3.50 (same interaction volume, ~$5/M input tokens)
Google Maps Geocoding	~$0.005 per request x ~20 stops = ~$0.10

Estimated total: ~$1.50-3.60 per active user per month, roughly 30-45x more expensive.

When our system is cheaper: at any usage level. For a student project scaling to hundreds of users, GPT-4o costs would become prohibitive quickly.
When our system becomes more expensive: if self-hosting a quantized 8B Llama model on a single GPU instance (~$100/month on cloud), we would break even at roughly 1,200-3,000 active users/month. Below that threshold, Groq API is cheaper. Above it, self-hosted inference wins on marginal cost.
Compute and storage: the backend is stateless Flask on Vercel serverless with no persistent CPU/RAM allocation. Groq inference is offloaded entirely to Groq's infrastructure. Trip data is small (~1KB per stop); 10,000 trips is approximately 50MB.

Production Readiness
Scaling plan (10,000 users/day):

- Stateless backend: Vercel auto-scales serverless functions horizontally with no configuration changes required.
- Groq API: At 10K users/day with ~5 AI calls each, peak load is ~50K calls/day, well within Groq's enterprise limits.
- Supabase: Would need to upgrade to the Pro plan ($25/month) for connection pooling (PgBouncer) and higher storage limits. Row-level security is already configured.
- Geocoding: Photon is a public API; at 10K users/day we should self-host the Photon instance (Docker image available) to avoid dependency on a free public service.

Rate limiting and abuse prevention:

- Current state: No per-user rate limiting on AI endpoints.

- Recommended: Implement per-user request limits (e.g., 20 AI calls/hour) via Supabase Edge Functions. The Groq API key is server-side only, never exposed to the frontend; all AI calls route through the Flask backend.

Privacy considerations:

- User trip notes are sent to Groq's API. Groq's terms of service state that data is not used for model training.
- No PII is required; users can use the app pseudonymously.
- Trip data is stored in Supabase with row-level security; users can only access their own trips.
- For stricter privacy requirements, self-hosted Ollama inference would eliminate third-party data exposure entirely.

Logging and monitoring:

- Current state: Basic request/response logging via Vercel's built-in function logs.
- Recommended: Log AI endpoint latency and token usage per request, alert if geocoding success rate drops below 80%, and use Sentry (free tier) for exception tracking on the Flask backend.